第 3-4 讲 • 动手：给列表加一个新列

> **模块 3 • 学会读代码** ｜ 建议用时 **40 分钟**（阅读 15 + 动手 25）
> 前置要求：完成 3-1 ~ 3-3

本节目标

读完并做完这一讲，你能：

1. **独立完成一次真实的代码修改**：在记录列表里加一列，显示"这是收入还是支出"的净额符号。
2. 掌握改代码前的**影响面分析**：动手前先想清楚"要碰哪个文件、改几处"。
3. 学会**"最小改动"原则**：为什么第一次改代码要改得越少越好。
4. 知道改完之后**怎么验证**：不只看"有没有效果"，还要看"有没有破坏别的"。
5. 体验一次完整的循环：**读 → 想 → 改 → 验证 → 收尾**。

一、本次任务

1.1 需求

给记录列表加一列「净额」，显示这笔账对余额的影响：

- 支出 → 显示负数（如 `-12.00`）
- 收入 → 显示正数（如 `+12,550.00`）
- 转账 → 显示 `–`（转账不影响总收入支出）

1.2 为什么选这个任务

理由	说明
真实有用	不是"玩具练习"，这个列确实能帮用户一眼看出方向
只碰一个文件	只需改 `RecordTable.tsx`，**不涉及数据库、不涉及 IPC**
能立刻看到结果	存盘就刷新，反馈即时
有"判断逻辑"	要区分三种类型，不是纯粹的复制粘贴

最后一条很重要：这个任务**刚好够难**——不难到让你卡住，也不简单到没收获。

二、动手前：影响面分析

2.1 先看现状

第一步永远是"看清楚现在是什么样"，而不是急着改。

```
cd "C:\Users\34123\Desktop\记账簿"
Get-Content src\renderer\src\features\records\RecordTable.tsx | Select-String "<th" | ForE
```

你会看到 9 个表头（这是真实的输出）：

行号	列
47	日期（条件显示）
48	类型
49	分类
50	账户
51	备注
52	收据
53	金额（`text-right`，右对齐）
54	操作

2.2 分析：要改几处

要加一列，必须改两处：

#	改哪里	为什么
1	`<thead>` 里的表头	不加表头，列就没有标题
2	`<tbody>` 里每一行的单元格	不加单元格，行里就没有内容

这就是"表格必须成对改"：表头和数据行是**两组独立的结构**，加列要同时改。

如果是真实的商业项目，这里就要小心了：列的顺序要对齐、宽度要分配、窄屏要考虑。**但作为第一次修改，我们只求"能正确显示"。**

2.3 为什么这次不用改数据库

一个重要的判断：为什么加"净额"列不用改数据库？

因为净额是"算出来的"：

```
净额 = 支出 ? -金额 : 收入 ? +金额 : 无
```

它不需要存储——数据（`amount` 和 `type`）**已经在数据库里了**，净额只是**换个方式显示。**

这个判断很关键：每次加功能前先问一句——

> **"这是新数据，还是已有数据的新显示方式？"**

- **新数据** → 要改数据库（迁移 + 仓储 + 类型，见 3-3 讲）
- **新显示方式** → 只改界面

本例属于后者，所以只需改一个文件。**这就是"影响面分析"的价值：它能帮你把 8 个文件的活缩成 1 个文件。**

三、动手

步骤 1：启动开发模式

```
npm run dev
```

等窗口出现，切到「记录」页，确认能看到账目列表。

步骤 2：读一遍你要改的文件

```
Get-Content src\renderer\src\features\records\RecordTable.tsx | Select-Object -Skip 40 -Fi
```

重点看两段：

① 表头（`<thead>`）：

```
<th className="text-right font-normal px-2 h-10 w-32">金额</th>
<th className="text-right font-normal px-4 h-10 w-24">操作</th>
```

② 数据行（`<tbody>` 里的一行）：

```
<td className="text-right px-2 py-2.5">
  <MoneyText cents={item.amount} type={item.type} signed strong />
</td>
```

读懂了什么：

- 每一列都是一个 `<th>` / `<td>`
- `className` 是样式（`text-right` = 右对齐、`w-32` = 宽度）
- `MoneyText` 是一个组件（2-4 讲学过），接收 `cents` 和 `type`

★ **一个关键发现**：金额那一格的真实写法是——

```
<MoneyText cents={item.amount} type={item.type} signed={item.type !== 'transfer'} strong /
```

注意 `signed={item.type !== 'transfer'}` **这一句。**它在说：

> "**只有不是转账时，才显示正负号。**"

这就是你要实现的那条规则，项目里已经写好了。 所以你加"净额"列时，完全可以**照抄这个写法**，而不是自己发明一套。

这说明一件重要的事：**你想加的功能，项目里很可能已经有一半了。** 先读代码再动手，能省掉大量重复劳动，而且**风格一致。**

步骤 3：加表头

在 `RecordTable.tsx` 里找到「金额」那一行的 `<th>`，**在它前面**插入一行：

```
<th className="text-right font-normal px-2 h-10 w-28">净额</th>
```

为什么放在金额前面：因为"净额"是金额的方向版，放一起更好读。**放在操作列前面**也行——但要**和下面的** `<td>` **顺序保持一致**（这是最容易错的地方）。

步骤 4：加数据单元格

在 `<tbody>` 里找到金额那个 `<td>`，**在它前面**插入：

```
<td className="text-right px-2 py-2.5 tabular-nums">
  {item.type === 'transfer' ? (
    <span className="text-muted">–</span>
  ) : (
    <MoneyText cents={item.amount} type={item.type} signed />
  )}
</td>
```

逐行解释：

代码	意思
`item.type === 'transfer' ? ... : ...`	如果是转账，显示 `–`；否则显示金额
`<span className="text-muted">–</span>`	一个灰色的破折号
`<MoneyText cents={...} type={...} signed />`	**复用现成组件**，`signed` 让它显示正负号
`tabular-nums`	让数字等宽，列对齐更整齐
`text-right`	右对齐（数字列的惯例）

注意一个关键设计：**转账不显示金额，而显示 `–`**。

为什么：因为转账**不是收入也不是支出**（2-3 讲说过，转账不计入收支口径）。如果显示 `+500.00`，用户会以为赚了 500。**显示 `–` 才是诚实的。**

这条规则不是我编的——你在项目的其他地方也能看到同样的处理（比如汇总页的统计就明确"转账不计入收支"）。**保持一致，是这个项目的一个好习惯。**

步骤 5：存盘，看效果

存盘，回到记账簿窗口。**dev 模式会自动刷新。**

预期看到：表格多了一列「净额」，支出显示 `-12.00`、收入显示 `+12,550.00`、转账显示 `–`。

如果没看到变化：

1. 确认存盘了
2. 确认终端里 `npm run dev` 还在运行
3. 看终端有没有报错（**很可能是我上面代码里的拼写错误**）

步骤 6：验证（这一步不能省）

很多人改完看到"有效果"就结束了，这是危险的。 正确的验证要问三个问题：

① 新功能对了吗？

对照三种类型逐个检查：

类型	应该显示	实际显示
支出	`-12.00`	?
收入	`+12,550.00`	?
转账	`—`	?

② 有没有破坏别的？

跑类型检查（这是最快的"有没有改坏"检查）：

```
npm run typecheck
```

预期：**0 错误。**如果有错误，先修掉再看效果。

③ 别的页面还正常吗？

点一下侧栏几个页面（日历、汇总、账户），看有没有异常。

为什么必须做第 ② ③ 步：因为你改的是一个**共享组件。实测一下它被谁用**：

```
Get-ChildItem src\renderer\src -Recurse -Include *.tsx | Select-String "RecordTable" | For
```

实际结果——有 4 个页面在用它：

文件	用途
`features\records\RecordsPage.tsx`	**记录页**（主用）
`features\search\SearchPage.tsx`	**搜索页**
`features\calendar\views.tsx`	**日历页**（日视图里列当天账目）
`features\recycle\RecycleBinPage.tsx`	回收站（只引用了相关工具函数）

所以你加的列，会在多个页面同时出现。

这通常是你想要的（各处保持一致），但**你必须知道这件事**——否则等你发现"搜索页怎么多了个净额列"，会以为是 bug。

改完之后至少检查两个页面：记录页 + 搜索页。

步骤 7：收尾

如果你打算保留这个改动：什么都不用做。

如果你想还原：

```
# 本项目已经接入 git（远程仓库在 GitHub），所以还原只要一条命令
git restore src/renderer/src/features/records/RecordTable.tsx
```

如果那时候项目还没有 git，还原就要**手动删掉你加的那两行**——很难保证删干净。

> **这就是版本控制存在的意义**：有了 git， `git restore 文件名` 一条命令回到上次提交，不用记自己改过什么。
> 现在这个项目已经接入 git，所以下面是更省事的做法。

下次你要做更大的改动时，先执行：

```
Copy-Item src\renderer\src\features\records\RecordTable.tsx "$env:TEMP\RecordTable.backup.
```

这就是"改前备份"的习惯——不是不信任自己，而是**成本极低、收益极高。**

四、学到了什么（经验总结）

4.1 一次完整的改代码流程

① 看清楚现状（读一遍要改的文件）
② 影响面分析（要改几处？会不会牵动别的文件？）
③ 最小改动（只改必须改的，别顺手重构）
④ 存盘 + 看效果
⑤ 验证三问：新功能对吗？类型检查干净吗？别的页面正常吗？
⑥ 收尾（保留还是还原？备份在哪？）

这六步就是模块 4 的全部内容。 你已经走了一遍。

4.2 三条重要经验

经验一：先找现成的工具

本次任务里，金额那一格**已经在用** `signed={item.type !== 'transfer'}` —— **你要的规则项目里已经写好了。**

如果没有先读代码，你可能会自己发明一套格式化逻辑——**重复造轮子，而且风格不一致。**

> **改代码前先问：这个功能，项目里是不是已经有一半了？**

经验二："加功能"和"改数据"要分清

- **新数据的显示** → 只改界面（本次任务）
- **要存新数据** → 牵动 3-3 讲那 8 个地方

分清这两者，能帮你把工作量估准。 大多数人低估的是后者。

经验三：共享组件的改动是"放大"的

`RecordTable` 被 **4 个地方**引用（记录页、搜索页、日历页、回收站），所以你改一处，**多个页面都变。**

> **改任何东西前，先搜一下"谁在用我"。**
> ```powershell
> Get-ChildItem src -Recurse -Include *.tsx,*.ts | Select-String "RecordTable"
> ```

这是模块 4 最重要的习惯：知道你的改动的波及范围。

4.3 关于"最小改动"

你可能会想：既然在改这个表格，不如顺便把列宽调好看点、把排序加上去……

不要。 第一次修改要遵守"**最小改动原则**"：

理由	说明
出问题了容易定位	只改了一处，报错必然来自这一处
容易还原	记不住改了哪五处，就还原不干净
验证简单	只需验一件事，不用同时验五件

"顺手改一下"是 bug 的头号来源。 等你熟练了（模块 6、7），再考虑"顺便优化"。

五、自测题

1. 给表格加一列，为什么至少要改两处？

2. 为什么"加净额列"不需要改数据库？什么情况下才需要改数据库？

3. 转账为什么显示 `-` 而不是数字？

4. 改完代码后，"验证三问"是哪三问？

5. 为什么改 `RecordTable` 会影响两个页面？这提醒你改代码前该做什么？

答案与解析

1. 因为表格是**两组独立结构**：`<thead>` 里的表头 和 `<tbody>` 里每一行的单元格。 → 只加表头不加单元格，列就是空的；只加单元格不加表头，就没标题。**而且顺序必须对应。**

2. 因为净额是**算出来的**（由 `amount` 和 `type` 推导），**不需要存储**——数据已经在数据库里了。 → 需要改数据库的情况：**要存"新的数据"**（比如给记录加"商家名称"字段）。判断标准：**"这是新数据，还是已有数据的新显示方式？"**

3. 因为**转账既不是收入也不是支出**（不计入收支口径）。显示数字会让用户误以为赚了或花了钱。 → 显示 `-` 才是诚实的，而且**和项目其它地方的处理保持一致**（汇总页也明确"转账不计入收支"）。

4. ① **新功能对吗**（对照预期逐个检查）；② **类型检查干净吗**（`npm run typecheck`）；③ **别的页面正常吗**（尤其是共享组件被多处引用时）。 → 很多人只做第一问就收工，这是危险的习惯。

5. 因为 `RecordTable` 是**共享组件**，实测有 **4 个地方**在引用它（记录页、搜索页、日历页、回收站），改一处多处都变。 → 提醒：**改代码前先搜"谁在用我"**（`Select-String "组件名"`），先知道波及范围再动手。

六、术语表

术语	一句话解释
影响面分析	动手前先搞清"要改哪些文件、会波及谁"
最小改动原则	只改必须改的，不顺手做别的优化
共享组件	被多个页面复用的组件，改动会被"放大"
`<thead>` / `<tbody>`	表格的表头部分 / 数据行部分
`tabular-nums`	让数字等宽显示，便于列对齐
`signed`	`MoneyText` 的参数，控制是否显示正负号
回归检查	改完之后确认"原有功能没被破坏"
改前备份	动手前复制一份原文件，便于还原
基线（干净状态）	动手前类型检查 0 错误的那个状态

七、本讲与项目的对应

- **你唯一改动的文件**：`src\renderer\src\features\records\RecordTable.tsx`（加了 2 处）
- **本讲不影响数据库、不影响你的记账数据**

- 如果你想保留这一列：什么都不用做；如果想还原：删掉你加的那两行
- 上一讲：3-3 类型与契约 ｜ **下一讲进入模块 4：4-1 改代码的安全流程**

八、模块 3 小结

讲	你学到的
3-1	**读代码心法**：带着问题、从入口追一条线；主进程的五层地图
3-2	**追数据流**：五站链路、为什么主进程要二次校验、SQL 占位符
3-3	**类型与契约**：类型只在写代码时有效、改字段的 8 处影响面
3-4	**动手改代码**：加一列、验证三问、最小改动原则

模块 3 结束时，你已经具备了"读懂任何一段代码"的能力：给你一个功能名，你能找到它的位置；给你一条数据，你能追出它的路径；给你一个改动，你能估出它的范围。

模块 4 要把这个能力变成"手" —— 从"看得懂"到"改得对"。

讲义版本：v1 ｜ 对应项目版本：0.1.0